

PRÁCTICA 5B

Daniel Cuevas Amodia y Diego Saro Gómez

1- Diseñar casos de prueba de caja negra basándose en la documentación de la interfaz `IConjuntoOrdenado`.

Para el diseño de los casos de prueba de caja negra, me he basado exclusivamente en la especificación de la interfaz `IConjuntoOrdenado`. He aplicado las técnicas de Partición Equivalente y Análisis de Valores Límite (AVL) para identificar las clases de entrada y reducir el número de pruebas a un conjunto mínimo representativo.

Método `add(E elemento)`:

Clases de Equivalencia (PE):

- CE1 (Válida): El elemento no existe en el conjunto.
- CE2 (Válida): El elemento ya existe en el conjunto.
- CE3 (No válida): El elemento es nulo.

Valores Límite: Inserción en los extremos y en el centro.

Casos de Prueba:

- **CP1 (PE):** Llamar a `add(5)` en un conjunto vacío. (Resultado esperado: devuelve `true` y `size=1`).
- **CP2 (AVL Inicio):** Llamar a `add(1)` en un conjunto `{3, 5, 7}`. (Resultado esperado: devuelve `true` y se inserta en el índice 0).
- **CP3 (AVL Fin):** Llamar a `add(9)` en un conjunto `{3, 5, 7}`. (Resultado esperado: devuelve `true` y se inserta al final).
- **CP4 (AVL Medio):** Llamar a `add(4)` en un conjunto `{3, 5, 7}`. (Resultado esperado: devuelve `true` y se inserta en el medio).
- **CP5 (CE2):** Llamar a `add(5)` en un conjunto `{3, 5, 7}`. (Resultado esperado: devuelve `false` y el conjunto no cambia).
- **CP6 (CE3):** Llamar a `add(null)`. (Resultado esperado: lanza `NullPointerException`).

Métodos `get(int index)` y `remove(int index)`:

Ambos métodos operan sobre un índice y lanzan `IndexOutOfBoundsException` si es incorrecto. Asumo para el diseño un conjunto de tamaño `N=3`.

Partición Equivalente (Índice): Rango válido ($0 \leq \text{index} \leq N-1$) y rangos inválidos ($\text{index} < 0$ y $\text{index} \geq N$).

Valores Límite: Los límites críticos se sitúan en 0 y `N-1` (válidos), y en `-1` y `N` (inválidos).

Casos de Prueba (aplicables a ambos métodos):

- **CP7 (AVL Válido Inferior):** Llamar a `get(0)` / `remove(0)`. (Resultado esperado: se retorna/elimina el 1er elemento).
- **CP8 (AVL Válido Superior):** Llamar a `get(2)` / `remove(2)` en un conjunto de tamaño 3. (Resultado esperado: se retorna/elimina el último elemento).
- **CP9 (AVL Inválido Inferior):** Llamar a `get(-1)` / `remove(-1)`. (Resultado esperado: lanza `IndexOutOfBoundsException`).
- **CP10 (AVL Inválido Superior):** Llamar a `get(3)` / `remove(3)` en un conjunto de tamaño 3. (Resultado esperado: lanza `IndexOutOfBoundsException`).

Métodos `size()` y `clear()`

Al no recibir parámetros, para estos métodos de estado aplico la técnica de conjetura de errores, evaluando los estados límite del conjunto (vacío y poblado).

Casos de Prueba:

- **CP11 (Estado límite):** Llamar a `size()` en un conjunto recién creado. (Resultado esperado: retorna 0).
- **CP12 (Estado nominal):** Llamar a `size()` tras añadir 3 elementos. (Resultado esperado: retorna 3).
- **CP13 (Transición):** Llamar a `clear()` en un conjunto con 3 elementos. (Resultado esperado: tras la llamada, `size()` retorna 0).
- **CP14 (Estado límite):** Llamar a `clear()` en un conjunto que ya está vacío. (Resultado esperado: no se produce error, `size()` sigue siendo 0).

2 - Caja blanca

Cobertura de sentencia (exige que cada línea de código se ejecute al menos una vez).

- Excepción nula: En CP6.
- While: En CP3.
- Return false: En CP5.
- Return true: En CP1, CP2 y CP4.
- Los métodos `get`, `remove`, `size` y `clear` son de una línea, así que se cubren con los demás CPs.

Por lo tanto, podemos garantizar un 100% de cobertura de sentencias con los test de caja negra.

Cobertura de decisión y condición (Cada expresión booleana compuesta se evalúe a true y a falsa y que cada una de las sub-condiciones se evalúen también de esa forma).

- **Decisión 1:** while (indice < lista.size() && elemento.compareTo...):

- Condición 1: indice < lista.size()

- Condición 2: elemento.compareTo(...) > 0

C1 = true y C2 = true: Ocurre en CP3.

C1 = true y C2 = false: Ocurre en CP2 o en CP5.

C1 = false: Ocurre en CP3 (Al llegar al final).

- **Decisión 2:** if (indice < lista.size() && ...):

- Condición 3: indice < lista.size()

- Condición 4: elemento.compareTo(..) == 0

C3 = true t C4 = true: Ocurre en CP5.

C3 = true y C4 = false: Ocurre en CP2 y CP4.

C3 = false: Ocurre en CP1 y CP3.

Corrección del código ConjuntoOrdenado:

1. Implementación del control de duplicados (CP5):
El código inicial no validaba si existían repetidos. Lo hemos solucionado añadiendo "if (comparacion == 0)", si se cumple la condición detenemos la ejecución, retornamos false y evitamos la inserción.
2. Corrección de la lógica de ordenación (CP2, CP3 y CP4):
La condición original "elemento.compareTo(...) < 0" era errónea. El índice solo avanzaba si el elemento a insertar era menor que el elemento actual, por ello las inserciones se hacían en posiciones incorrectas.
Lo hemos solucionado aislando la lógica de evaluación. Ahora el índice solo se incrementa si el nuevo elemento es mayor que el elemento evaluado "else if (comparacion > 0)".
3. Refactorización para mejorar la eficiencia y legibilidad:
En el código inicial, si se hubieran añadido más comprobaciones en el while, se habría tenido que invocar compareTo varias veces por iteración.
Ahora hemos extraído el resultado de la comparación a una variable local. De esta forma, se invoca al método de comparación una vez por iteración.